DevOps Foundation



Introduction

DevOps Foundation

- **The demand for DevOps has been increasing within the market**
- **DevOps Engineer ranks in the top five of all tech salaries, commanding an average pay of $111,683.** Dice's "2019 Tech Salary Report,"
- **DevOps Offers a Definite Career Path that Promises Steady Growth**

HOWEVER

People-related factors tend to be the greatest challenges—not technology. George Spafford, Senior Director Analyst at Gartner

Introduction

Exin DevOps Foundation

Who Should Attend this training ?

Introduction

DevOps Foundation

- **IT consultant:**
- ✓ **System or Cloud Engineer / Administrator**
- ✓ **Database developer, tester or administrator**
- **IT Manager or Project Manager**
- **Beginners: student or IT specialist wishing to discover the world of DevOps**

Training goals

- **Exin DevOps Foundation**
- Understand what the DevOps movement is
- Know the challenges of the DevOps profession
- Master the values, principles and practices of DevOps
- Get an overview of automation tools
- Prepare for and pass the certification



Exam Details

Duration :

60 minutes

Is Exam open book :

Support not allowed

Number of questions:

40

Percentage of success:

65% (26 correct answers or more)

SUMMARY

- DevOps: General Overview
- DevOps principles and values
- Traditional methods vs DevOps
- DevOps practices
- DevOps application
- DevOps and tools
- Bonnus



Objectives

By completing this section, you will be able to:

1. **Describe the historical development from the waterfall to Agile and Scrum to DevOps.**
2. **Describe the technological and methodological origins of DevOps.**
3. **Define DevOps, what isn't and its benefits for business and IT.**



Introduction

What is DevOps ?

QUESTION : WHAT IS DEVOPS ?

1 DevOps Overview

DevOps – General overview

Problematic

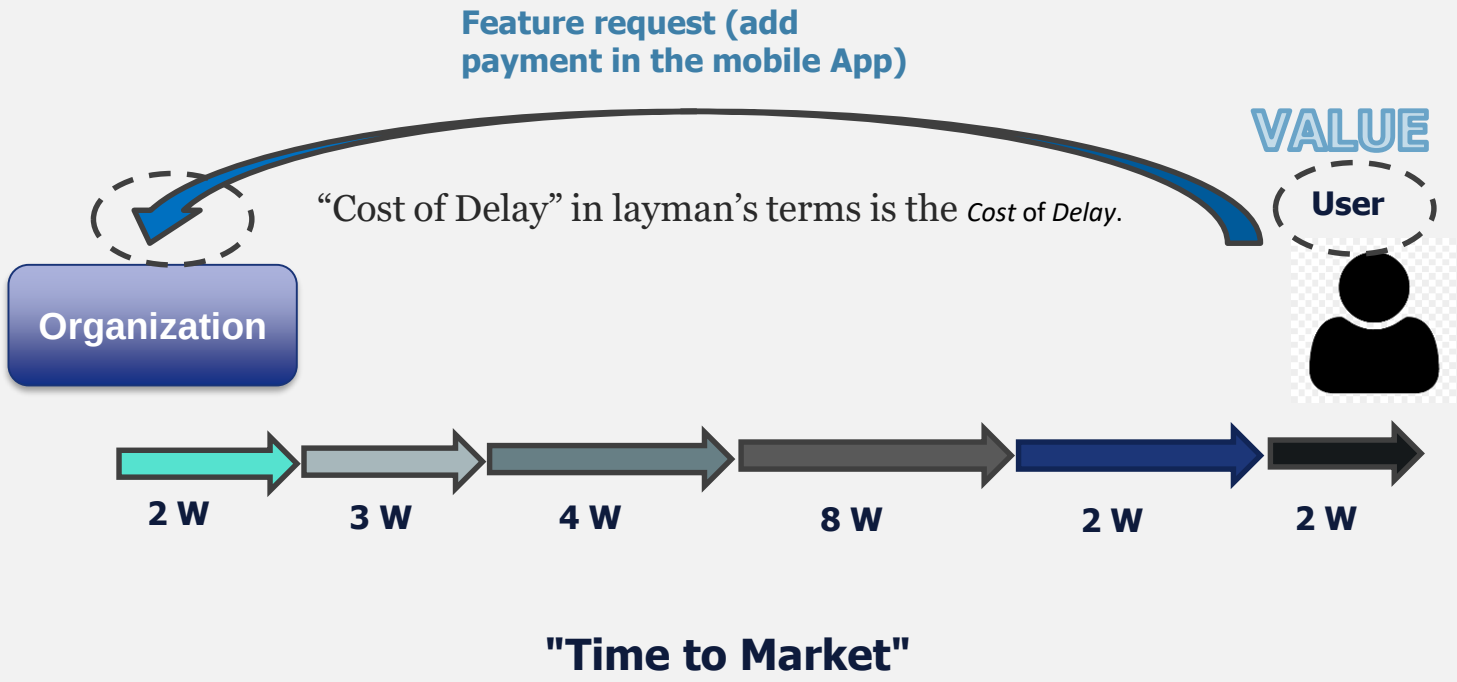
Let's set the problem !

Today we are experiencing a true merger between business and IT.Any business, regardless of its industry, relies on IT services. Responding to increasingly changing and increasingly complex needs depends on its ability to deliver quality IT services at ever faster pace.

The key indicator of agile business: "Time to Market"!

DevOps – General overview

Time to Market



DevOps – General overview

Cost of Delay

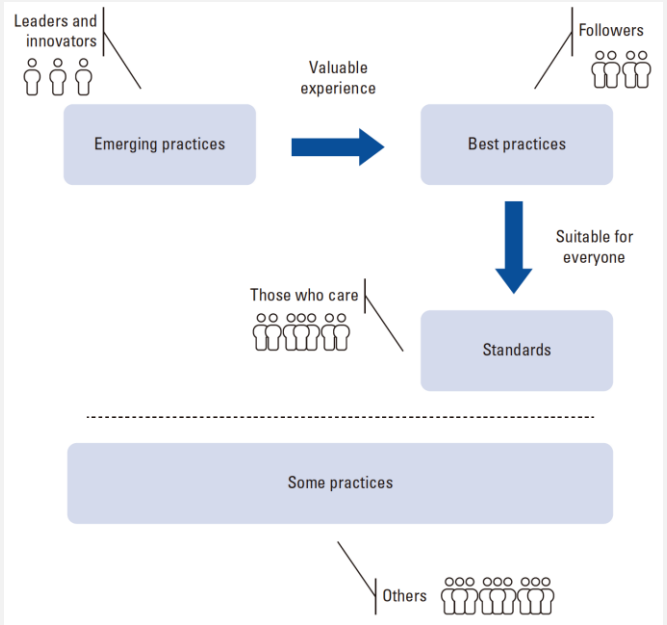
“Cost of Delay” in layman’s terms is the **Cost of Delay**.

More simply, it is the answer to the question: "What would it cost us if this was delayed by 1 month?". Or, alternatively, "what would it be worth to us if we could get this 1 month earlier?"

DevOps Origins

DevOps – General overview

DevOps origins



1980 …

ITSM
ITIL
COBIT
CASCADE
AGILE
SCRUM

2007

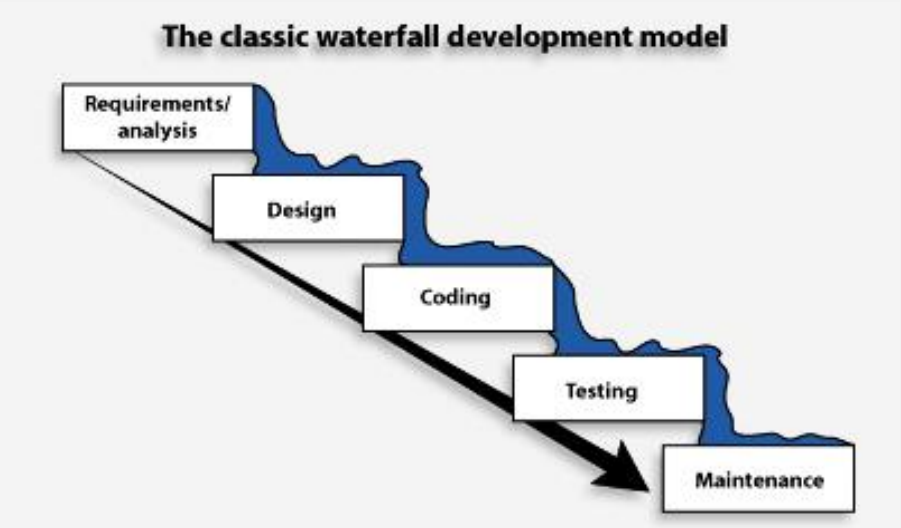
DEVOPS

KEEP C.A.L.M.S. AND DO DEVOPS

DevOps – General overview

From Waterfall to Agile to Cloud Computing

➢ A - Waterfall



- In the 20th century, Waterfall Model was the main methodology for software development.
- In the 1990s, the internet accelerated the need for quick launches from years to months.

DevOps – General overview

From Waterfall to Agile to Cloud Computing

➢ B - AGILE

In 2001 ‘17’ experts believed that the traditional (Waterfall Model) no longer corresponded to the constraints and requirements of rapidly changing organizations. Agile values, principles:

- A working software
- Continuous delivery
- Collaboration between Business and IT
- Employee motivation
- Agility
- Simplicity
- Self-organization
- Technical excellence



Creation of the Agile manifesto

DevOps – General overview

From Waterfall to Agile to Cloud Computing

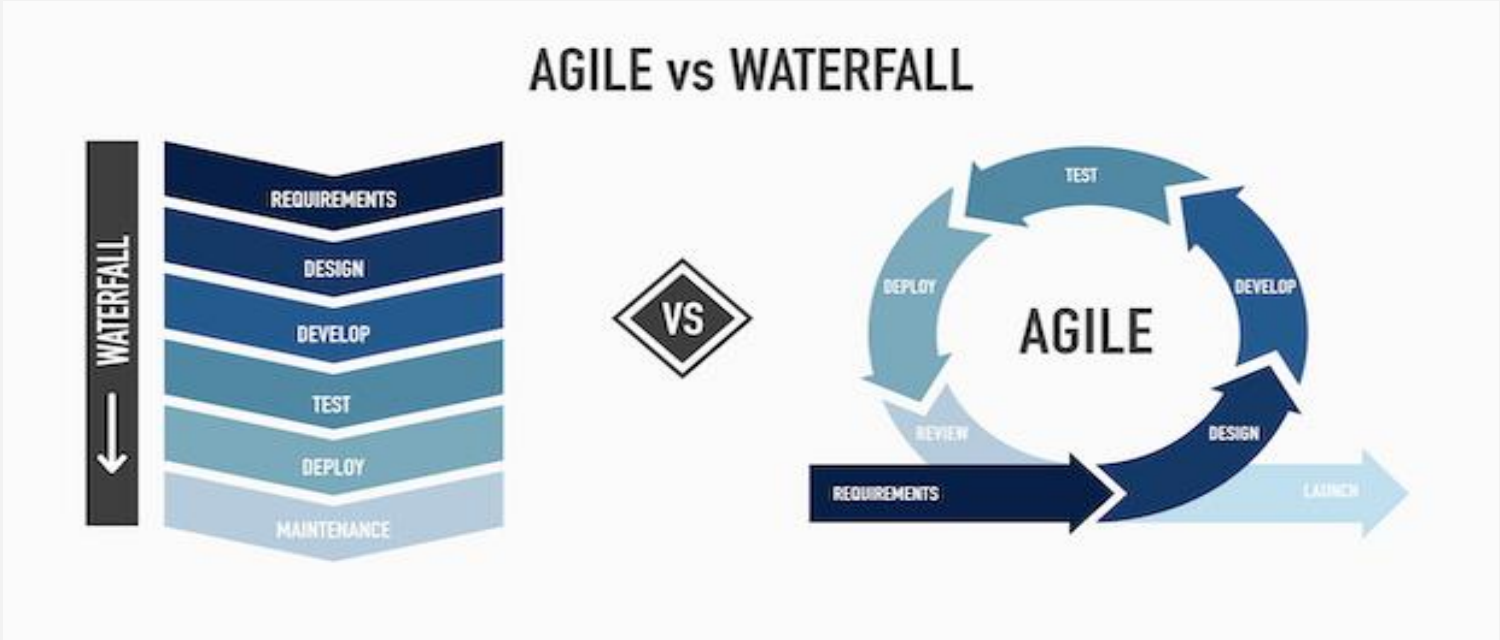
- AGILE MANIFESTO

Individuals and interactions	over	Processes and tools
Working software	over	Comprehensive documentation
Customer collaboration	over	Contract negotiation
Responding to change	over	Following a plan

That is, while there is value in the items on the right, we value the items on the left more

DevOps – General overview

From Waterfall to Agile to Cloud Computing



DevOps – General overview

DevOps Origins

➢ **LEAN SOFTWARE DEVELOPMENT :**

The 7 types of waste:

1) Defects or bugs (Defects in Lean Manufactoring)
2) Task switching (Transportation in Lean Manufacturing)
3) Extra features (Over-production in Lean Manufactoring)
4) Handoffs (Motion in Lean Manufactoring)
5) Delays (Waiting in Lean Manufactoring)
6) Partially done work (inventory in Lean Manufactoring)
7) Relearning (extra processing in Lean Manufactoring)

DevOps – General overview

DevOps Origins

➢ **LEAN SOFTWARE DEVELOPMENT :**

Definitions:

- **Jidoka (autonomation)** is the first pillar of the Toyota Production System. It's automation with a human touch, or how to automate simple, repetitive operations, while retaining human control to orchestrate all that tooling, and handle complex situations.
- **J-I-T (Just in Time)** is the second pillar of the Toyota Production System. It is the flow to deliver just in time (neither before nor after) the requested product, in the requested quantity.
- **Kaizen (continuous improvment)** consists of drawing lessons from problems, enriching our "system" and this through a structured approach and proven analysis tools (PDCA (Plan, Do, Check, Act), 5 whys, diagram Pareto etc). We all want to improve, Kaizen helps us structure and make these improvement efforts effective.

DevOps – General overview

DevOps Origins

➢ **LEAN SOFTWARE DEVELOPMENT :**

- Completed code not checked into a repository, undocumented code, untested code ➔ Delays the delivery of value and leads to quality issues

= Partialy done work

- Database administrator has to wait two days to have the server and software installed ➔ Delay to delivery of value

= Delay

- Revisiting the documentation each time we need to write a deployment script ➔ each time we have to learn again the how to ?

= Relearning

DevOps – General overview

From Waterfall to Agile to Cloud Computing

- C - INFRASTRUCTURE AS A CODE

```
{
  "Description" : "Create an EC2 instance running the Amazon Linux 32 bit AMI.",
  "Parameters" : {
    "KeyPair" : {
      "Description" : "The EC2 Key Pair to allow SSH access to the instance",
      "Type" : "String"
    }
  },
  "Resources" : {
    "Ec2Instance" : {
      "Type" : "AWS::EC2::Instance",
      "Properties" : {
        "KeyName" : { "Ref" : "KeyPair" },
        "ImageId" : "ami-3b355a52"
      }
    }
  },
  "Outputs" : {
    "InstanceId" : {
      "Description" : "The InstanceId of the newly created EC2 instance",
      "Value" : { "Ref" : "Ec2Instance" }
    }
  },
  "AWSTemplateFormatVersion" : "2010-09-09"
}
```

DevOps – General overview

From Waterfall to Agile to Cloud Computing

- C - INFRASTRUCTURE AS A CODE

Virtualization

- Efficient use of material resources

Cloud Computing

- VPN to send private data packets over shared channels with security, privacy and quality of service
- Large vendors have made virtual resources accessible and reliable

DevOps – General overview

From Waterfall to Agile to Cloud Computing

Cloud computing and virtualization

- Virtualization
  - fast provisioning of environments
  - reduced time to deploy software
  - easier to offer 'as a service' infrastructure
  - standardizing hardware



- Cloud computing
  - fast access, easily scalable
  - deploy to a completely standardized stack
  - no need to worry about configuring or maintaining testing, staging, or production environments, or virtual machine images

DevOps – General overview

From Waterfall to Agile to Cloud Computing

Resources offered by commercial providers have become affordable and reliable; they also assured the necessary level of security.



The customers' attitude to the clouds and their use has changed from *"someone else is controlling my hardware somewhere"* to *"I have an infrastructure that I manage remotely"*.

DevOps – General overview

From Waterfall to Agile to Cloud Computing

➢ C - INFRASTRUCTURE AS A CODE

Cloud Computing

According to NIST (The US National Institute of Standards and Technology) The essential characteristics of Cloud Computing are:

- ✓ On-demand self-service
- ✓ Broad network access
- ✓ Resource pooling
- ✓ Rapid elasticity
- ✓ Measured service

Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

This cloud model is composed of five essential characteristics:

- **On-demand self-service:** A consumer can unilaterally provision computing capabilities, such as server time and network storage, automatically as needed without requiring human interaction with each service provider.
- **Broad network access:** Capabilities are available over the network and accessed via standard mechanisms that promote use by heterogeneous thin or thick client platforms (such as mobile phones, tablets, laptops, and workstations).
- **Resource pooling:** The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. There's a sense of so-called *location independence,* in that the customer generally has no control or knowledge over the exact location of the provided resources but may be able to specify location at a higher level of abstraction (by country, state, or data center, for example). Examples of resources are storage, processing, memory, and network bandwidth.
- **Rapid elasticity:** Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be appropriated in any quantity at any time.

- **Measured service:** Cloud systems automatically control and optimize resource use by leveraging a metering capability at a level of abstraction that's appropriate to the type of service (storage, processing, bandwidth, or active user accounts, for example). Resource usage can be monitored, controlled, and reported, providing transparency for both the provider and consumer of the utilized service.

DevOps – General overview

From Waterfall to Agile to Cloud Computing

- C - INFRASTRUCTURE AS A CODE

Resources offered by commercial providers have become affordable and reliable; they also assured the necessary level of security.



The customers' attitude to the clouds and their use has changed from *"someone else is controlling my hardware somewhere"* to *"I have an infrastructure that I manage remotely"*.

DevOps – General overview

From Waterfall to Agile to Cloud Computing

First, the emergence of new methods of interacting with end customers and the application of agile development techniques.

+

Second, New technologies for infrastructure management:

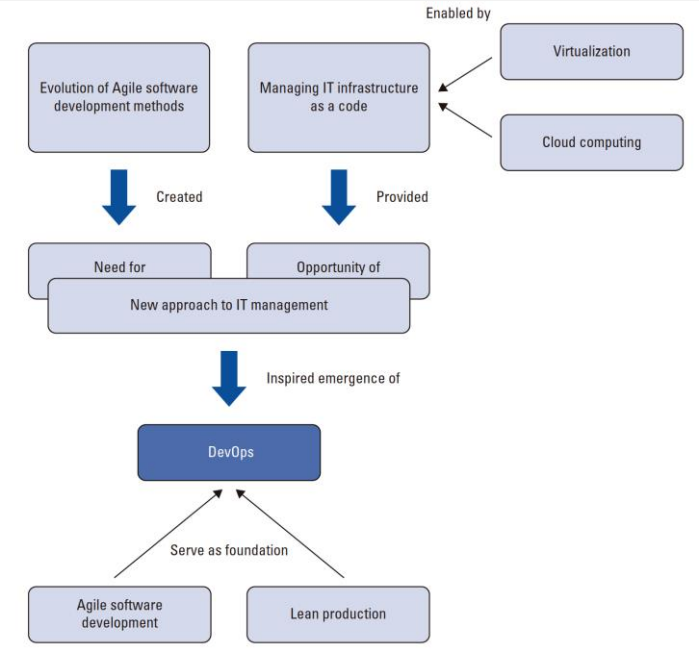
- Virtualization
- Cloud computing

=

It has become possible to **organize** IT work differently.

DevOps – General overview

Definition of DevOps



- DevOps :
  - **Development** (the teams in charge of projects / application development) **+ Operations** (the teams in charge of run and operation)
  - The very essence of DevOps is to think not only about software development, but also **the entire value chain.**



DevOps – General overview

Definition of DevOps

- By grouping together the different terms of several definitions given on the web, DevOps is:

- A movement of professionals inherited from 2 philosophies

LEAN SOFTWARE DEVELOPMENT

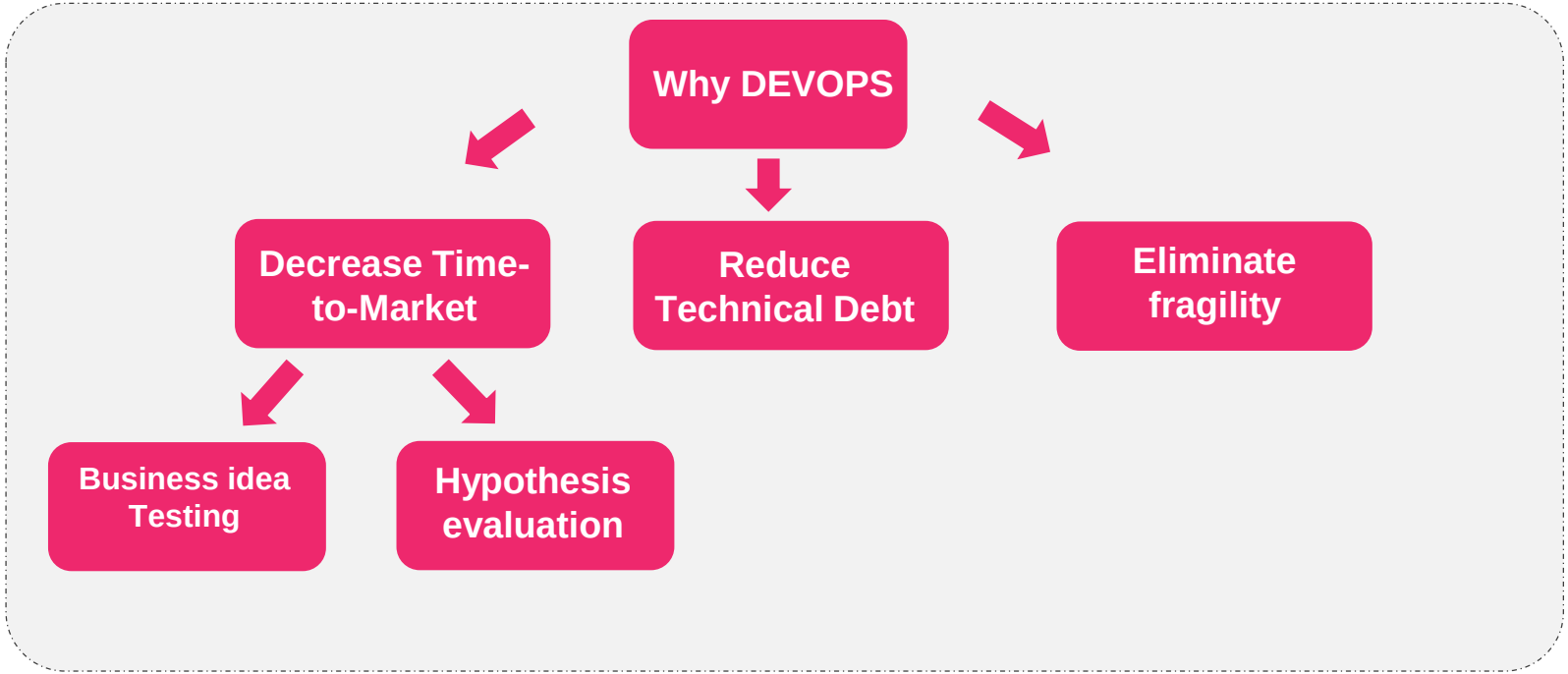
AGILE

- A culture ;
- An agile process across the entire chain (from development to production)
- A new technical approach
- Another human approach
- A response to new issues such as massive deployment and regular deployment.

DevOps Benefits

DevOps – General overview

DevOps Benefits

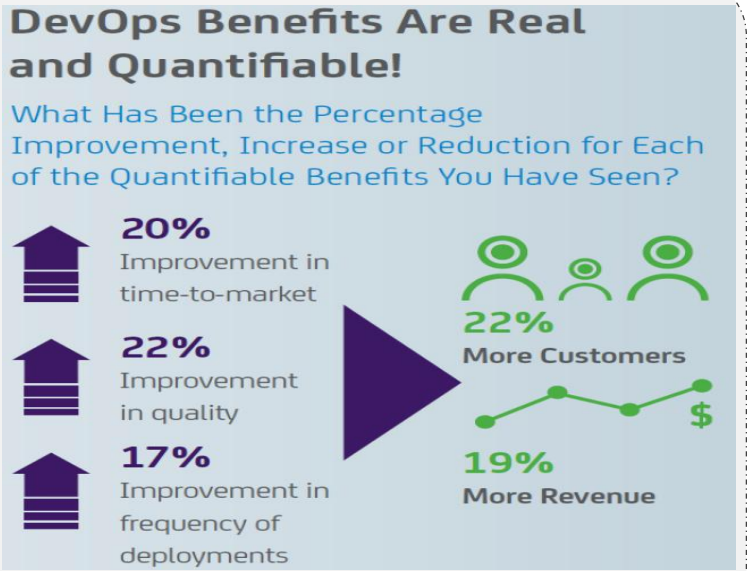


DevOps – General overview

Benefits : Time To Market

Decrease Time-to-Market

- Product size reduction.
- Reduction of losses in the deployment of a change into production.
- Continuous improvement.
- Elimination of wastes :
  - Verification of builds
  - Early quality check
  - ...
- Autonomous teams
- Automating



Source: CA study "What smart businesses know about devops". Panel: 1,300 IT decision-makers in 21 countries Available at http://aka.ms/devopsca

DevOps – General overview

Benefits : Technical Debt



The debt occurs when a programmer chooses a non-optimal way to solve a problem in order to shorten the development time.

DevOps – General overview

Benefits : Technical Debt

Reduce Technical Debt

*Debt arises when **a team member chooses a suboptimal way to solve a problem in order to reduce development time**. the non-optimal solutions accumulated lead to a progressive deterioration of the product, consequently, a degradation of the quality of the product.*

*Failure to comply with the design, whether intentional or not, results in additional costs in the future. **These are the interests**. This is why we speak of technical debt, to show the analogy with debt in corporate finance. This implies that it is better to pay off the debt one day rather than keep paying interest over and over.*
When we code as quickly as possible and in a non-optimal way, we contract a technical debt that we reimburse throughout the life of the product in the form of increasingly long development time and more and more frequent bugs.

DevOps – General overview

Benefits : Technical Debt

Reduce Technical Debt

The elimination of technical debts created previously (knowingly or accidentally) is planned in the same way as the creation of new functionalities.

DevOps strongly recommends using the "face issues as often as possible" practice to prevent a "stagnation" of issues that everyone knows, but nobody can get their hands on.

DevOps – General overview

Benefits : ~~Fragility~~

Eliminate fragility

Ironically, the most important and most profitable systems for businesses are the most fragile. Reducing the fragility of these systems is extremely difficult due to the ***high risks of business disruption****,* ***zero tolerance to downtime****, and the constant stream of new changes and improvements related specifically to these systems.*

In DevOps, the code and the system are still functional at all times and if the next change disrupts their performance, it is immediately rollbacked and the system continues to function properly.
IT systems need to react independently and quickly to detect a failure and restore normal operation, ideally so that end users don't notice and data --of course-- is not lost.

DevOps – General overview

Benefits : ~~Fragility~~

Eliminate fragility

“Anti-fragility is a property of systems that get stronger when exposed to stressors, shocks, volatility, noise, errors, faults, attacks or failures. “

Netflix Chaos Monkey



DevOps Myths

DevOps – General overview

Frequently expressed misconceptions

Myth or Reality: "DevOps is nothing more than the continuation of Agile ideas"

Based largely on Agile, DevOps **extends** Agile development ideas to Agile IT production, to the whole organization, to the whole process, to the entire value chain.

Achieving the benefits of DevOps **requires larger cultural changes in the business** than the adoption of Agile.

The **goals** set for DevOps aren't just about speeding up delivery - it's also about reducing technical debt and eliminating fragility.

DevOps – General overview

Frequently expressed misconceptions

Myth or Reality: "DevOps is a superman capable of coding, creating tests, deploying environments and managing infrastructure."

DevOps is a profound change in IT services, which cannot be achieved by hiring multiple DevOps engineers or inviting DevOps experts.

The ability to implement a software delivery pipeline does not guarantee success. DevOps isn't just about using tools. These are the profound organizational changes that can be supported by these tools.

DevOps – General overview

Frequently expressed misconceptions

- DEVOPS is
- Neither a product
- Neither a framework
- Nor a method

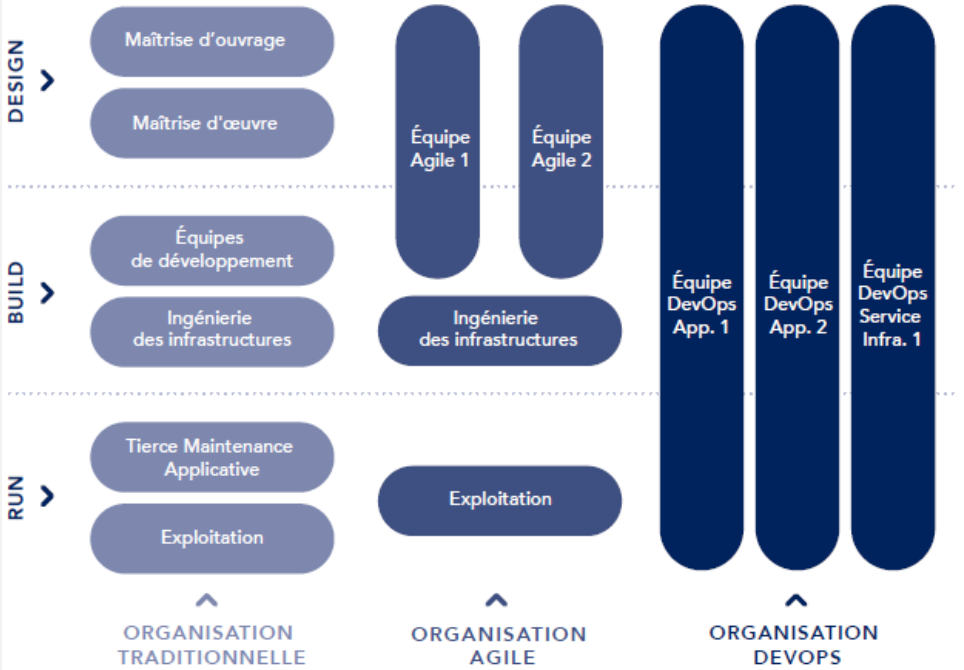
Has not been the subject of a manifesto

No clear consensus: the founder did not define DEVOPS (Patrick DEBOIS)

DevOps Organization

DevOps – General overview

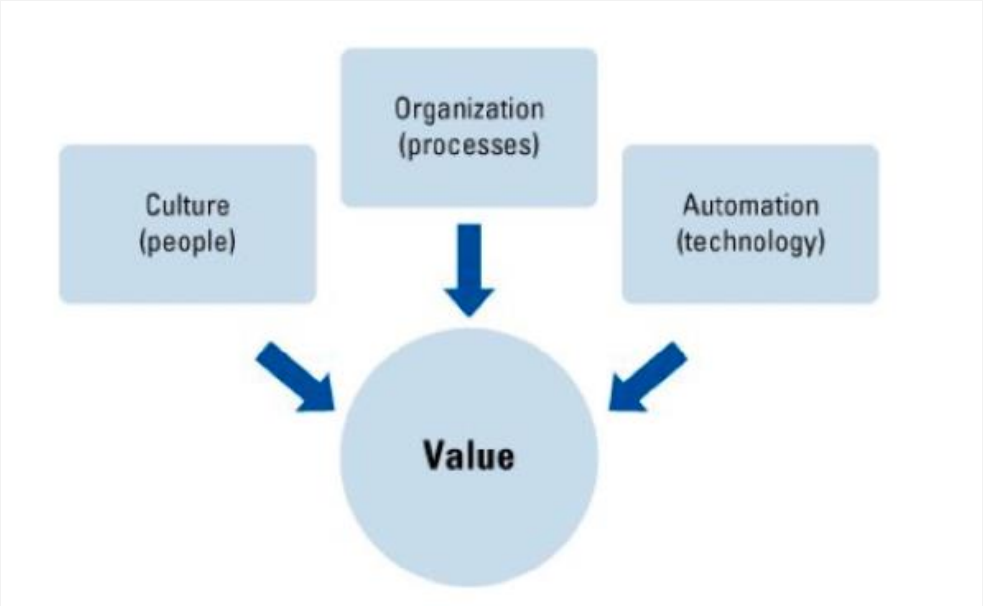
Organizational change



DevOps Definition

DevOps – General overview

The Definition



DevOps – General overview

The Definition

DevOps

is an evolution of the ideas of Agile software development methodology and Lean Manufactoring applied to the end-to-end IT value chain,

that enables companies to make profits with modern information technologies

due to cultural, organizational and technical changes.

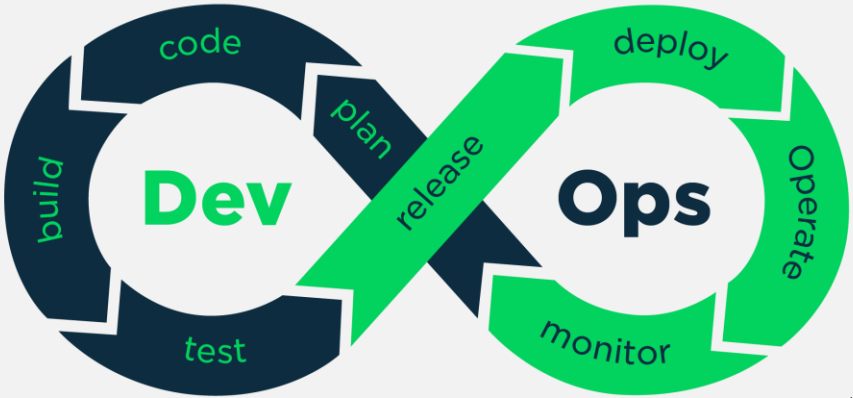
2 The Principles of DevOps

The principles and values of DevOps

The principles of DevOps

➢ The key principles

- Value Stream Mapping
- Deployment pipeline
- Version control.
- Configuration management
- Definition of Done



Value Stream Mapping

The principles and values of DevOps

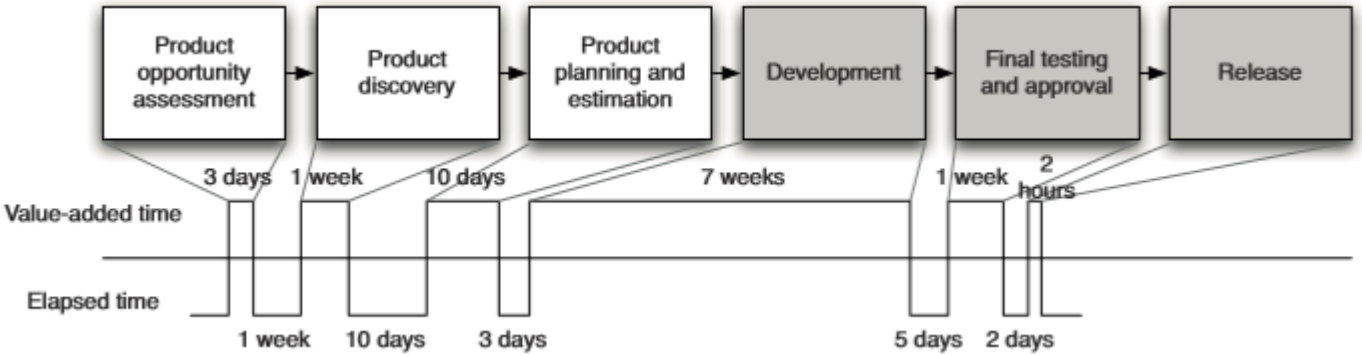
The principles of DevOps

With a pencil and pad in hand, go to the place where a customer request comes into your organization. You goal is to draw a chart of the average customer request, from arrival to completion. Working with the people involved in each activity, you sketch all the process steps necessary to fill the request, as well as the average amount of time that a request spends in each step. At the bottom of the map, draw a timeline that shows how much time the request spends in value-adding activities and how much in waiting states and non-value-adding activities.

Lean Software Development: An Agile Toolkit "Mary and Tom Poppendieck's"

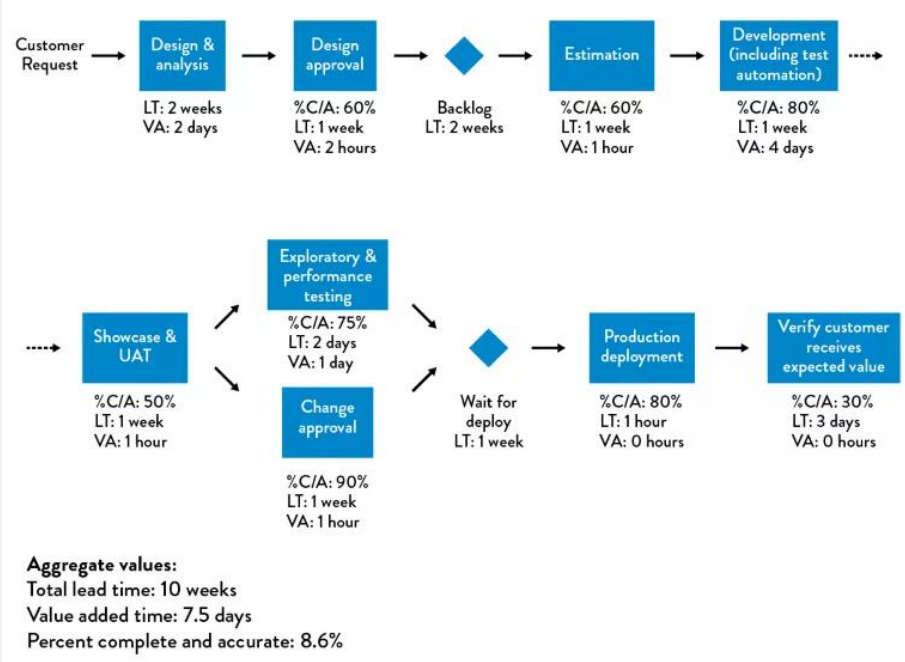
The principles and values of DevOps

The principles of DevOps



The principles and values of DevOps

Value Stream Mapping



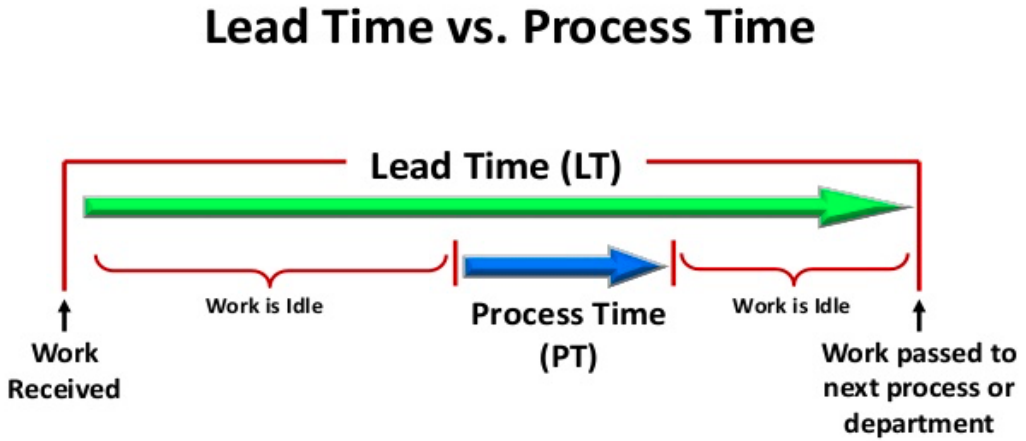
- Lead Time (LT)
- Process Time (PT)
- Percent Complete and Accurate (%C/A)

This percentage is obtained by asking downstream stages the percentage of time they receive “usable as is” work, without needing to clarify, add, or correct the information received in the previous step. Only the downstream step can evaluate the % C/A from the previous step.

The principles and values of DevOps

Value Stream Mapping

In the Lean Value Stream Map, Process Time (PT) is the time it takes to do the work. Lead Time (LT) includes the Process Time and everything else outside of that for that step. In the case of software development this can include time for ideation, QA testing, deployment time, etc.



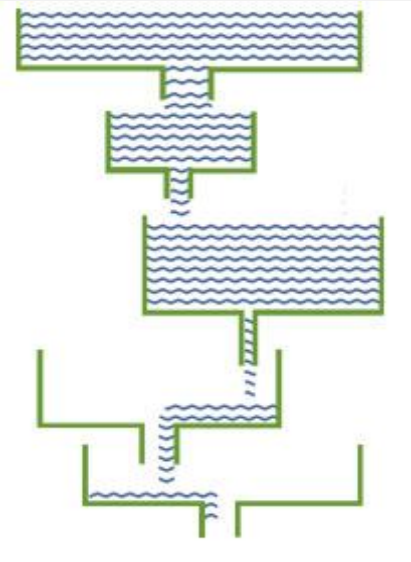
Lead Time = Elapsed time; Throughput time; turnaround time
Process Time = Touch time; work time; cycle time

The principles and values of DevOps

Value Stream Mapping

- Theory Of Constraints. (ToC)

> “Since the strength of the chain is determined by the weakest link, then the first step to improve an organization must be to identify the weakest link”
>
> — Eliyahu M. Goldratt, The Goal: A Process of Ongoing Improvement



The principles and values of DevOps

Value Stream Mapping

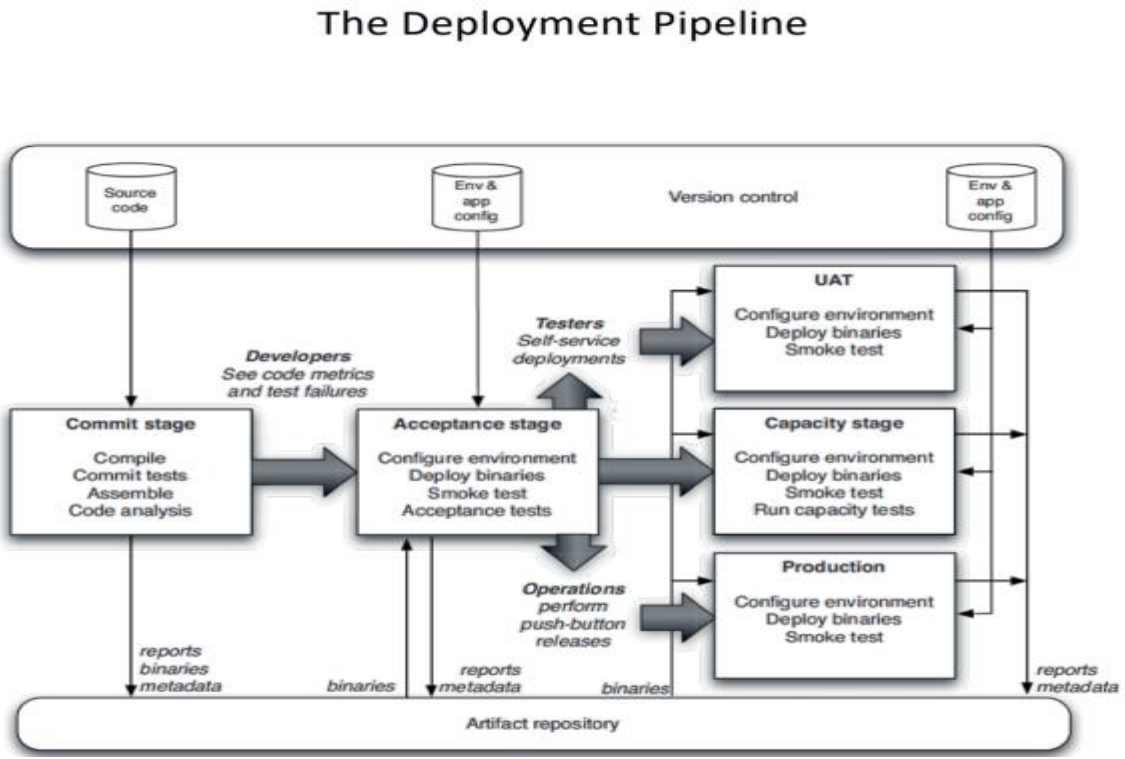
➢ Theory Of Constraints. (ToC)

- The Theory of Constraints provides an iterative improvement process in five steps, which aims to focus efforts on the constraint alone.
- The five steps are as follows:
  - Identify the constraint (the bottleneck).
  - Exploit the constraint; improve the use of its capacity.
  - Subordinate all processes to constraint.
  - Increase the capacity of the constraint if it is relevant.
  - Start over at step 1 if the constraint has changed.

Deployment pipeline

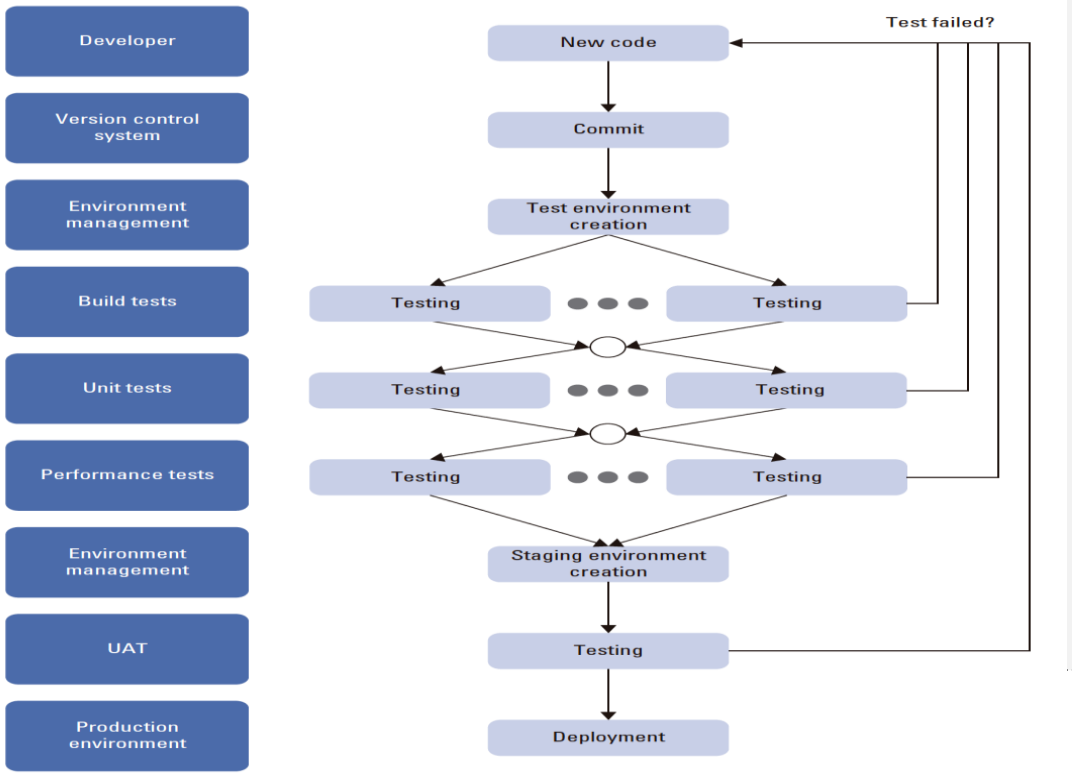
The principles and values of DevOps

The principles of DevOps



The principles and values of DevOps

Deployment pipeline



The principles and values of DevOps

Deployment pipeline

- **A modern process using a parallel pipeline**
- Allows you to produce results much faster.
- Saves resources by not starting the next steps before the end of the previous ones.

- **Ensures product quality**

- Modifications that do not work well do not reach production and the system is still in working order.

- **Accelerates the delivery of changes to the production environment**

- By maximizing the automation of each step.

- **Constantly leaves records in audit logs**

The principles and values of DevOps

Deployment pipeline

The implementation of the deployment pipeline introduces the following challenges :

1. Exessive enthusiasm for automation at the expense of ideology (process, people and culture) leads to creation of remarkably automated pipeline that nobody uses.

2. Initially, there are not enough pre-developed tests to ensure steady operation of the pipeline.

3. In the target state, there are so many tests that the passage of a change throught the pipeline takes too long and requeires significant computational resources, especially in case of a large amount of small changes.

The principles and values of DevOps

Many believe that the name 'pipeline' is taken by analogy with an assembly line, for example, in a car manufacturing plant. Others believe that the word 'pipeline refers to a liquid or other substance flowing through the pipes, and the deployment pipeline should follow this analogy. Both above opinions are inaccurate.

As the authors of the term, Jez Humble and David Farley[32], explain, the idea originated from the pipelining used in modern processors, where performance improvements cannot be achieved solely by increasing the clock frequency. The architectural solution used is the parallel execution of instructions, which initially come in sequence. To do this, the processor must 'guess' the results of processing in a parallel flow, 'assuming' that they will be as required to perform calculations in the current flow. If not, the results of the calculations will be discarded. The time lost due to this 'unlucky guess' is more than compensated by the acceleration for those cases when the 'guess' was correct.

So, a properly implemented deployment pipeline allows to make development and testing independent of each other in time: it is assumed that testing will be successful, so you can proceed to the next batch of work. The same logic is applied to the parallel testing.

The principles and values of DevOps

Deployment pipeline

- The three DevOps processes

Continuous Integration: is a build and test oriented process consisting of compiling, testing and deploying on an integration environment. The goal is to test as often and as much as possible the non-regressions of the deliverable to detect bugs as early as possible. Most of the work is done by test tools. Deployment on the integration platform becomes simple and can be achieved by developers without involving operations.

The principles and values of DevOps

Deployment pipeline

- The three DevOps processes

Continuous Delivery: is an integration and production process. The goal is to test and deliver an application at each stage of its life cycle (acceptance, pre-production, production). This step is carried out after validation of the tests carried out in integration. The test phase corresponds to the functional tests of the deliverable. The transition from one state to another is fully automated, which is why the deliverable must be made up in such a way that it can be deployed in production as soon as it is put into acceptance.

The principles and values of DevOps

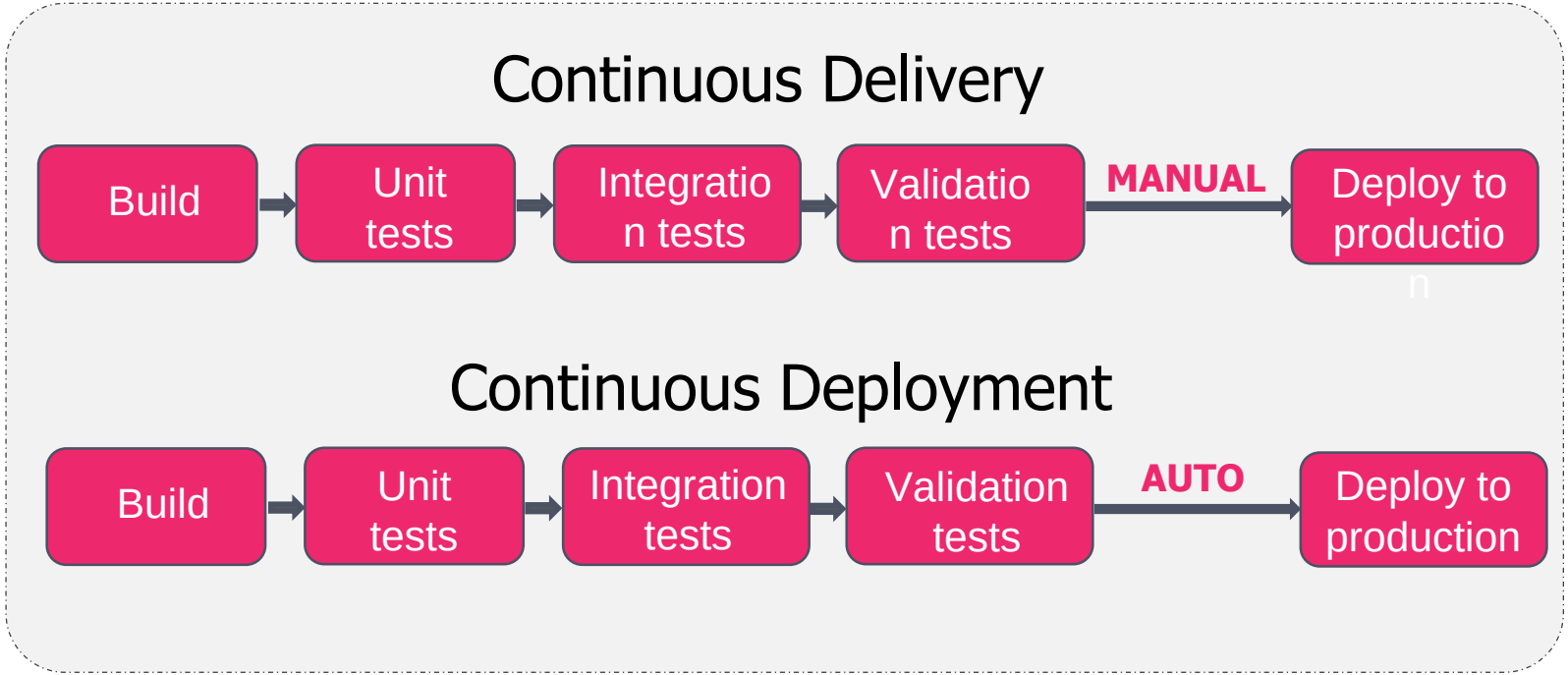
Deployment pipeline

- The three DevOps processes

Continuous Deployment: is a production process. The goal is to compile, test and deploy an application in production. Continuous Deployment requires that the Continuous Integration and Continuous Delivery processes have been successfully completed. Deployment is done with a simple “click on button". After deployment, it should be possible to measure any impacts using performance measurement and monitoring tools. If something goes wrong, an automated rollback process can be performed.

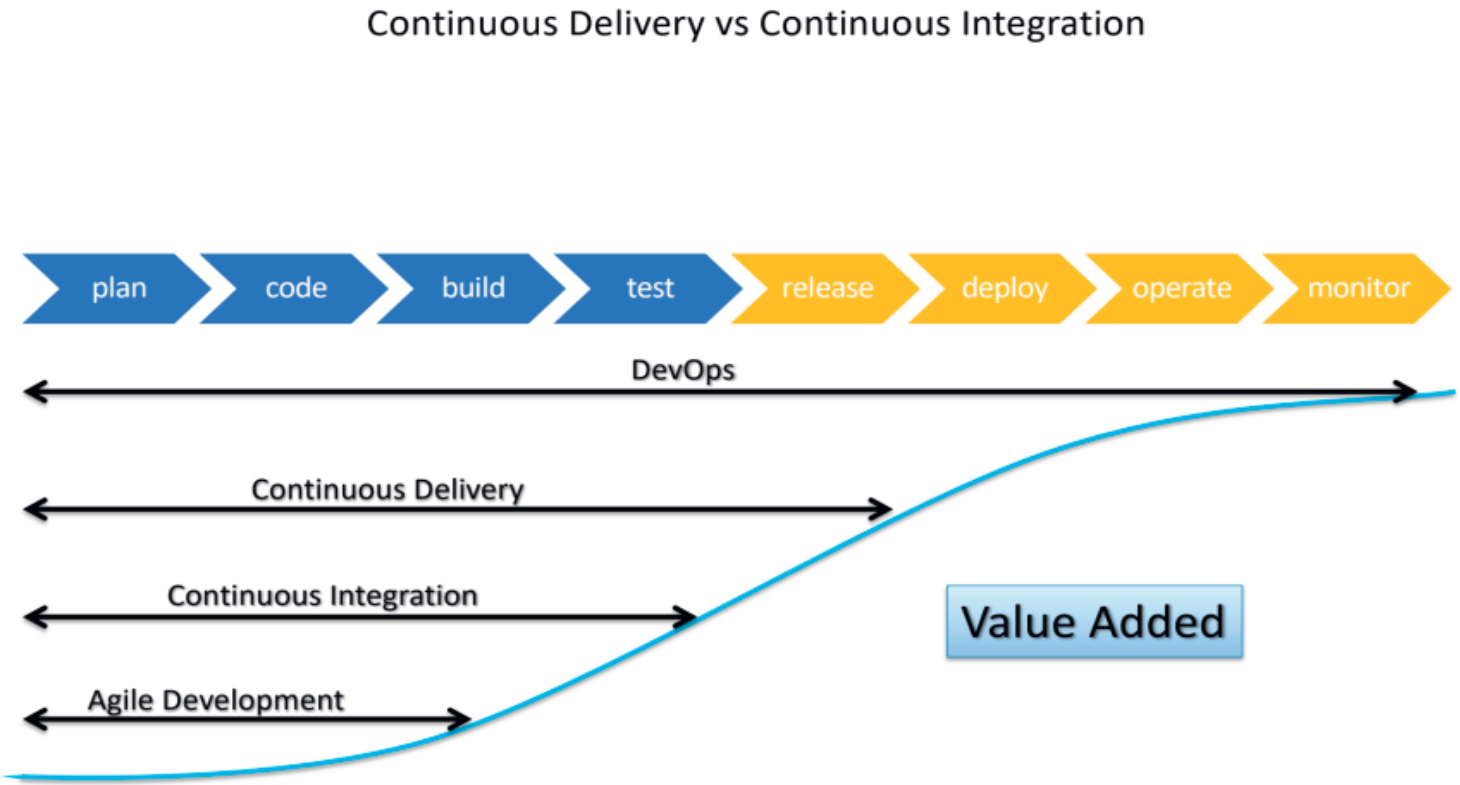
The principles and values of DevOps

Deployment pipeline



The principles and values of DevOps

Deployment pipeline



Version Control

The principles and values of DevOps

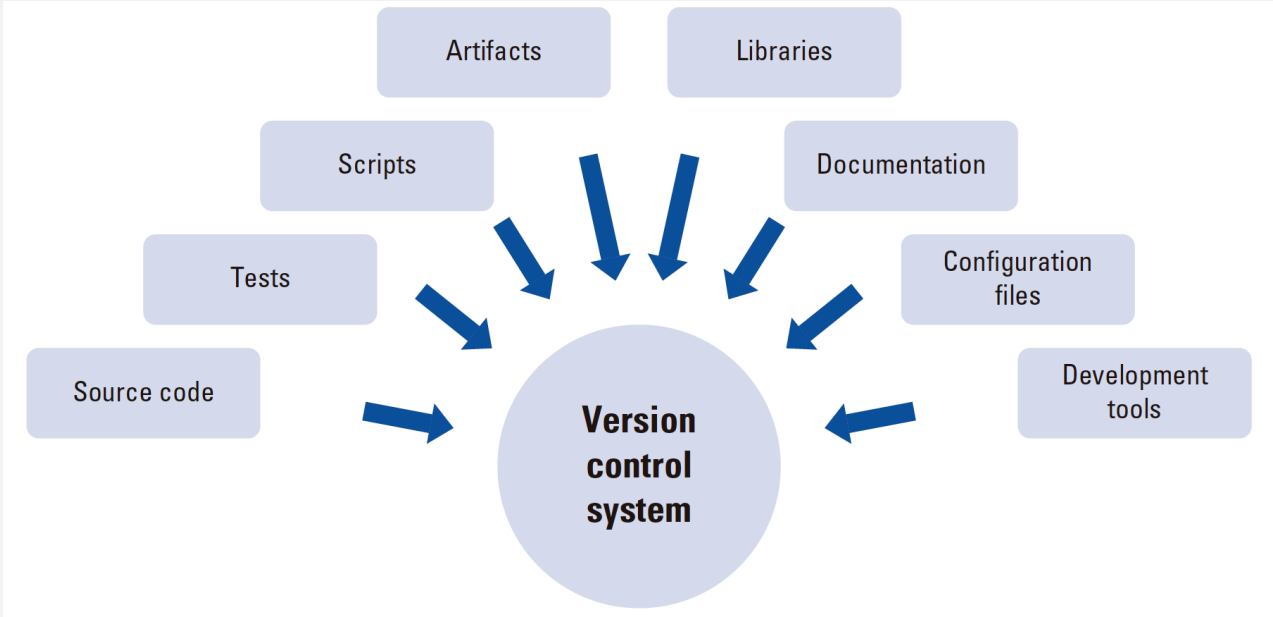
Version Control

This is about storing not only the source code, but also everything related to the whole pipeline :

- tests,
- scripts to create and modify databases,
- build scripts,
- environment creation scripts (including the development environment),
- deployment scripts,
- artifacts,
- libraries, documentation, configuration files,
- even development tools, such as compilers, IDEs, etc.

The principles and values of DevOps

Version Control



The principles and values of DevOps

Version Control

- **Profits**

- ✓ The ability to determine what was changed, when and by whom.
- ✓ Ability to restore the system at any time, including returning the failed system to a guaranteed working state with minimal effort.
- ✓ Allow a team member to freely delete unnecessary files and documents, without the risk of accidental loss of important information or products

Configuration Management

The principles and values of DevOps

Configuration Management

In DevOps any changes to the environment (infrastructure that host the application) should be done by a script stored in the version control tool.

The creation of environments is done automatically when the deployment pipeline is running.

This principle requires a complete reorganization of IT support work and operations. Indeed, administrators no longer have the right to modify anything in the production environment in the usual way.

The principles and values of DevOps

Configuration Management

- **Benefits**
- DevOps completely restructures the management of the production environment (as well as any other environment).
- Any modification of an environment can only be done by scripts stored in the version control system.
- The creation of environments is done automatically when the deployment pipeline is running.
- This principle requires a complete reorganization of IT support work and operations. Indeed, administrators no longer have the right to modify anything in the production environment in the usual way.

The principles and values of DevOps

Configuration Management

➢ Advantages

- ✓ DevOps configuration management provides the same benefits that you get with full version control, but **the main beneficiaries are now the people working in Operations.**

- ✓ Now that all changes are checked, the system can be quickly restored to a stable state. If key members leave the organization, knowledge is not lost, etc.

Definition of Done

The principles and values of DevOps

Definition of Done

- Definition of Done

It's not when someone has done their part of the job, but when (WHEN) the customer received or began to receive the value they expected.

This means that the entire value stream has been fully tracked to the production environment; only then will the work be considered finished.

In agile, a finished request: developed, tested and delivered.



Summary

The principles of DevOps

3 Traditionnal vs DevOps

Traditionnal vs DevOps

Continual improvement

- Frequent release

Traditionnal	DevOps
Big batch sizes	Small batch sizes
Scheduled delivery	Continuous delivery
Many days / weeks	Communicate weekly / daily
Exthensive resources	Efficient use of resources
High effort	Regular effort
backups	Automation
Documentation	Continuous
Manual	

Traditionnal vs DevOps

Continual improvement

- The most important differences

Traditionnal	DevOps
- Release - A release is a group of new features deployed together in the production environment. - Delivery schedule - IT decision	- Deployment - Partial or complete delivery of a new functionality for users - Deployed as soon as the tests are successful - Business decision

Traditionnal vs DevOps

Automation

- Pipeline Automation

The environments required for the deployment pipeline are created automatically by scripts.

These environments are automatically released after use.

Fast pipeline operation requires maximum test automation

Deployment and distribution, the final stages of the pipeline, are also done automatically.

Traditionnal vs DevOps

Continual improvement

- Continual improvement

All identified defects must be eliminated immediately.

For example, if a script that runs the deployment pipeline does not work properly, it should be fixed immediately.

DevOps recommends repeating problematic steps as often as possible, unlike the traditional practice that issues can be deferred.

This will help to better understand how they should be improved and to adjust the working method accordingly.

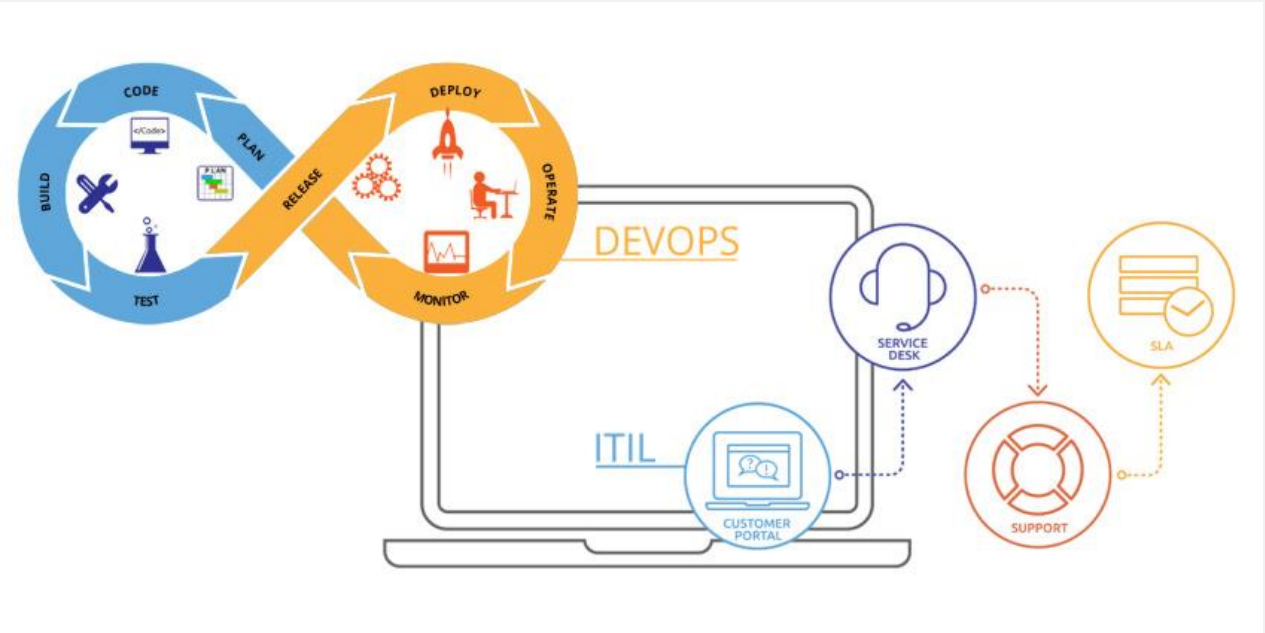
Operations methodologies (IT Service Management)

- ITSM supports ongoing availability and maintenance of services to ensure the delivery of value and desired service outcomes
- The Service Lifecycle:
  - Service Strategy
  - Service Design
  - Service Transition
  - Service Operation
  - Continual Service Improvement
- Most widely used methodology is ITIL®



Traditionnal vs DevOps

DevOps and ITSM



4 DevOps Practices

DevOps Practices

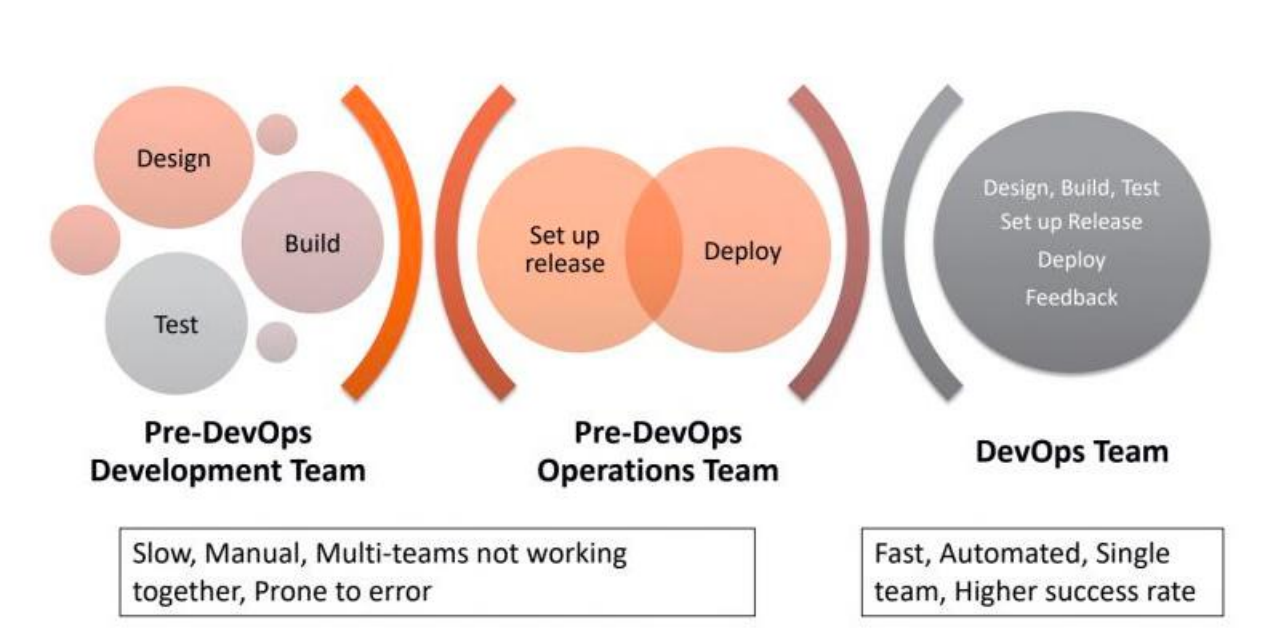
- DevOps practices are:

  - Diverse team
  - Visualizing work
  - Small batch sizes
  - Supporting innovation
  - Identifying ways to deal with bottlenecks

Diverse team

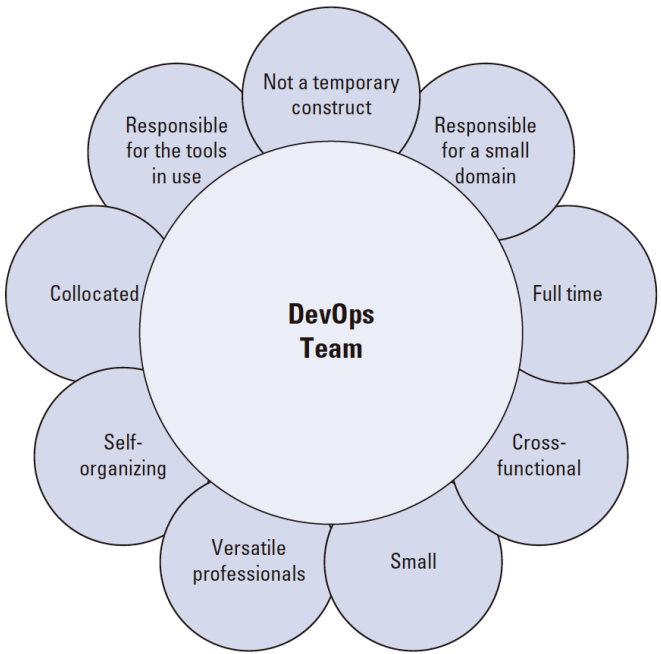
DevOps Practices

DevOps Team



DevOps Practices

Autonomous and diversified team



DevOps Practices

Diverse team

Recruit a mix of individuals

Gender, sexuality, religion,
race, colour,
age, ability,
education level,
experience

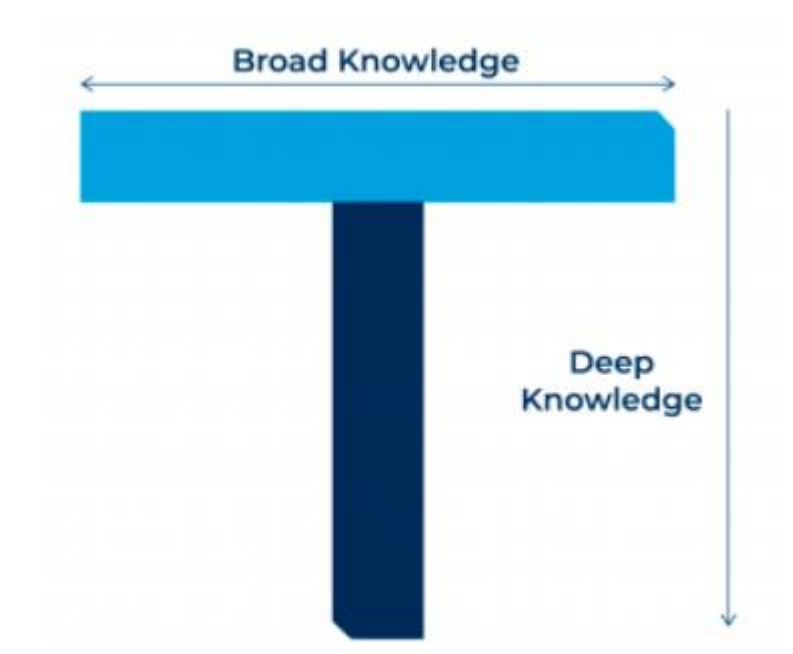


Ensure working conditions and methods adapt to the diversity

Benefits: Increases different view points and ability to challenge, question, innovate and feedback that generates **Learnings**

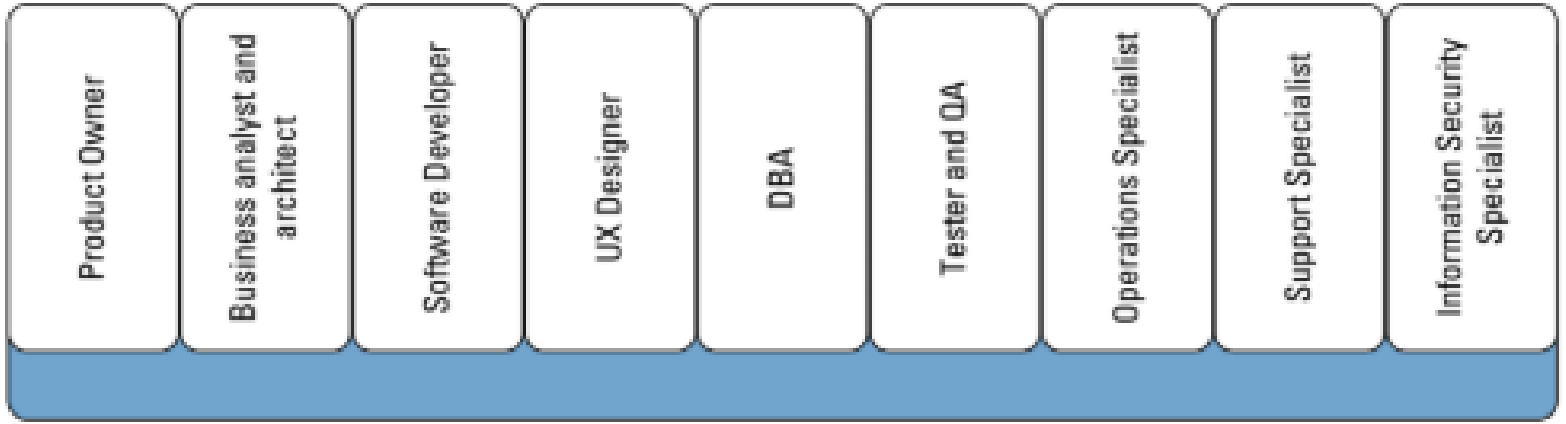
DevOps Practices

T-Shaped profile/skills



DevOps Practices

DevOps team example



DevOps Practices

A new DevOps profiles examples

Cloud Architect - A person with extensive hands-on experience building cloud infrastructures and understanding what they need to include to support various types of applications and services in production.

Site Reliability Engineer (SRE) - He is the DevOps specialist who focuses on ensuring stable performance and uninterrupted availability of high-load applications on large-scale systems.

DevOps tools administrator - one of the main DevOps roles as it is responsible for all DevOps tools and the cloud platform used for deployments.

Visualizing work

DevOps Practices

Visualizing work

The work visualization can support the deployment pipeline and help find answers to the following questions:

- What step that does not allow the rest of the chain to function effectively?
- Where are the resources already or almost exhausted?
- What tasks are blocked, so that they have no chance to be completed in this iteration?
- What remains to be done for the task that has not been completed?
- If we don't have time to do all of the work accepted in this iteration, what part of the task is worth trying to finish to get the maximum outcome possible?

DevOps Practices

Visualizing work



- Managers, sales staff, designers, developers, operators, and customer support staff are one team and share all the business information on visual boards
- Visual control means “Does everybody easily understand the situation by just looking at the boards without explanations?”
- Enables fast view of progress and identification of issues

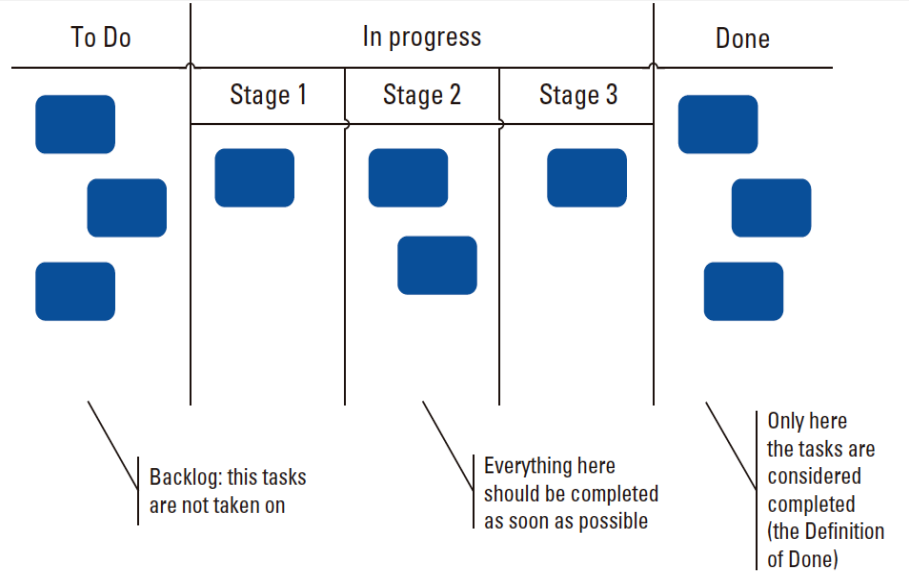
DevOps Practices

Visualizing work

➢ KANBAN

A Kanban board allows you to create
A "pull system":

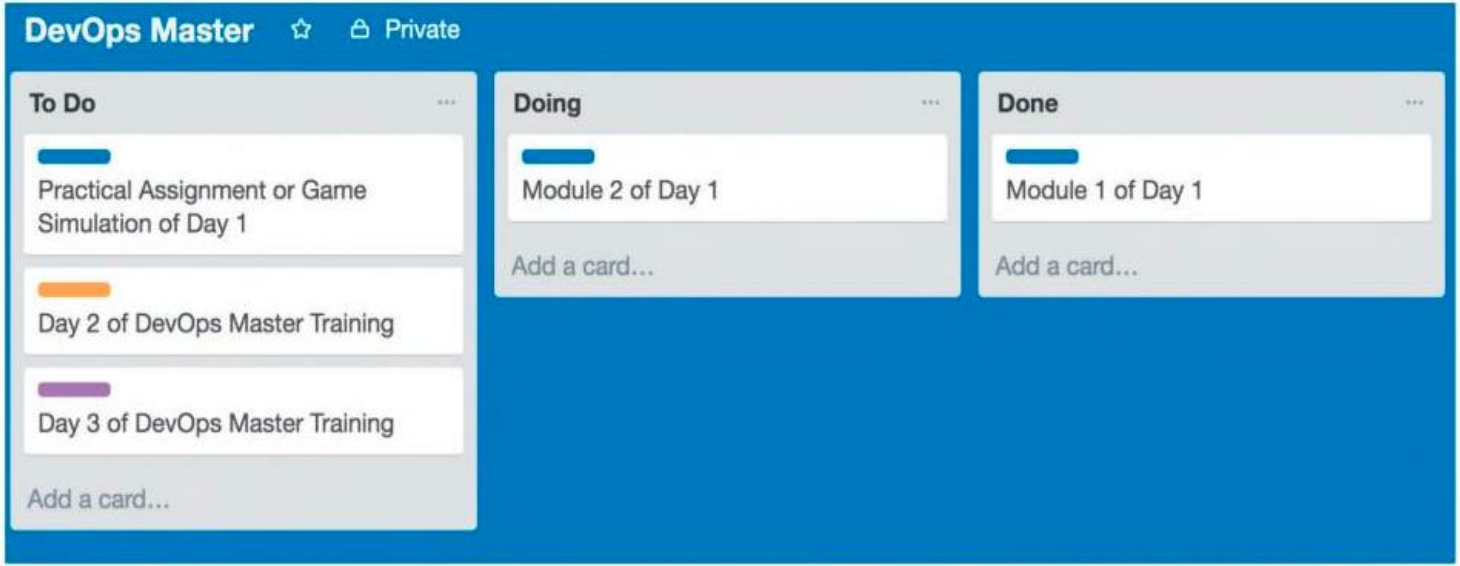
- Visualize the workflow
- Measure and optimize the duration of flow
- Reduces downtime
- Reduces the need for coordination
- Limit the work In Progress (WIP)



DevOps Practices

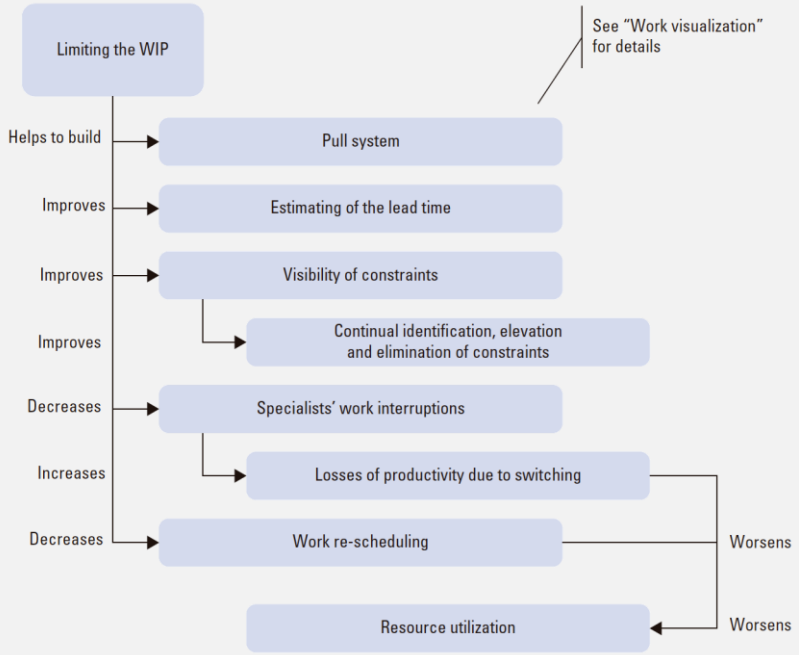
Visualizing work

The Kanban board



DevOps Practices

Visualizing work



Small batch sizes

DevOps Practices

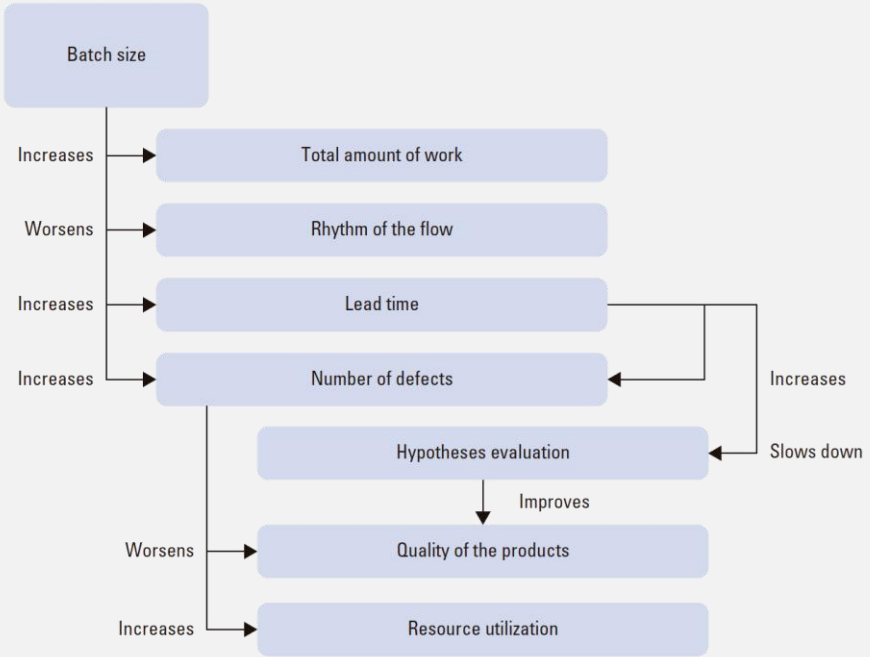
Small batch sizes

Reducing the size of release batches is the second way to improve the workflow (WIP).

Small batches pass through the system faster and with less variability, which promotes faster learning and deployment. This usually means focusing more attention and increasing investments in infrastructure and automation. It also reduces the transaction cost of each batch.

DevOps Practices

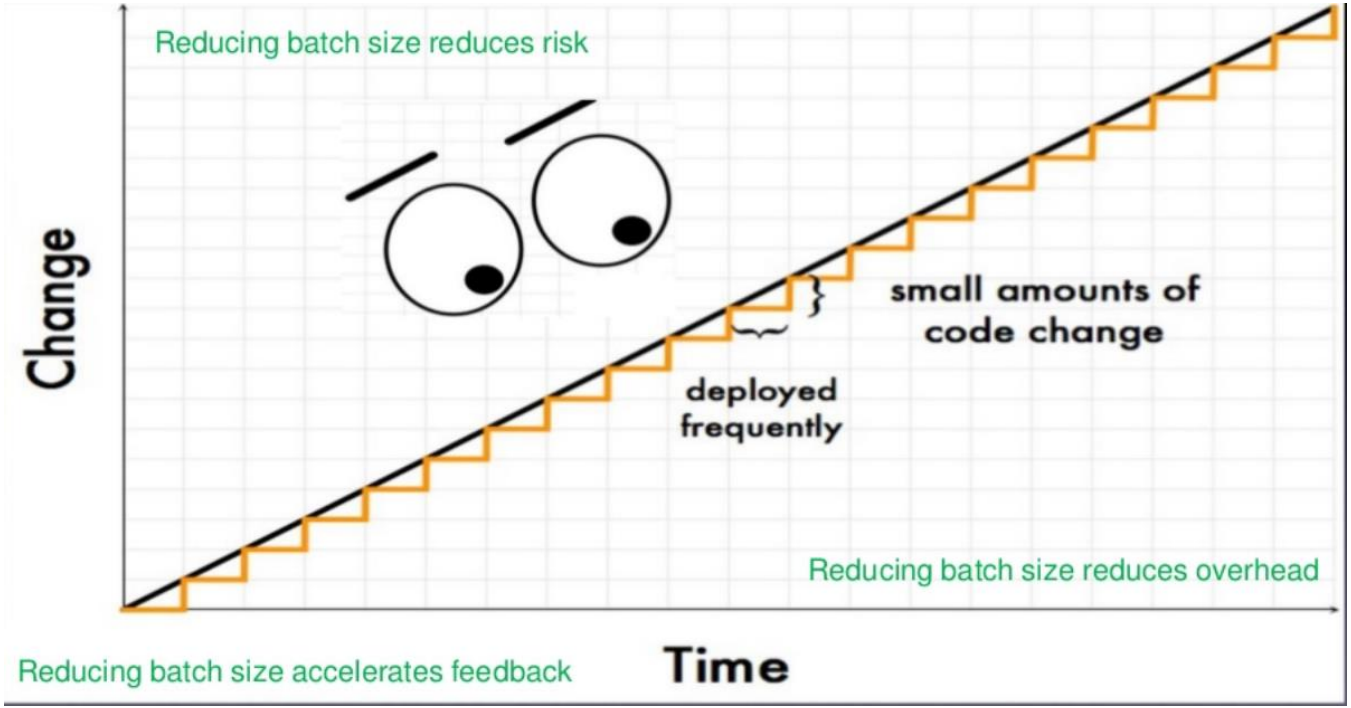
Small batch sizes



DevOps Practices

Small batch sizes

Anti-Fragility :



Supporting innovation

DevOps Practices

Supporting innovation

- Encourage innovation

Some companies start by allocating a certain proportion of working time to improvement and innovation (20%).

A special budget allocated to the exploration of new technologies and the creation of new products and tools.

Select interesting ideas, invest limited resources in some of them.

Dealing with Bottlenecks

DevOps Practices

Identify ways to deal with bottlenecks

➢ Dealing with Bottlenecks

A good, smooth deployment pipeline without delays is not achieved overnight and requires considerable effort.

Using visualization tools and WIP limits, it is possible to identify bottlenecks in the chain.

Of all known bottlenecks, there's one that causes the most lag - that's the one to focus on.

Change the working method in order to capitalize on the bottleneck and exploit it to the fullest.

Find a way to eliminate the bottleneck.

5 Applicability and Difficulties

Applicability and Difficulties

Objectives

- Characterize situation in which DevOps is feasible
- Identify conditions that make DevOps interesting for the business
- Identify the lack of readiness to adopt DevOps
- Characterize monolothic IT infrastructure and architecture as a limitation for adopting DevOps
- Clarify risks and solutions to the use of COTS in strategic business lines
- Characterize the need for a flexible mindset to change and innovation
- Recall that DevOps can start small and can be built up from there

DevOps Application

Application

Applicability



➢ DevOps is not...

DevOps is not a magic solution for all "problems".

Don't start DevOps practices if...

- The company only participates in a limited part of the value stream.
- The company does not view technology as a key enabler of its business.

Application

Applicability



- DevOps should be of interest to companies when the following conditions are met:

- All **other methods** adopted for increasing efficiency **no longer give significant results.**
- Its main activity is highly **dependent** on information technology.
- The information technology used by this organization has **a high rate of change.**
- The main activity (business) requires **rapid changes** to test new ideas or hypotheses: to remain competitive

Application

Applicability



- An investment
- Companies that want to significantly reduce technical debt or eliminate IT infrastructure fragility:
- The time and effort required to obtain significant and continuous benefits from DevOps is very important and should be considered an investment.

Limitations

Application

Limitations



- DevOps is not very suitable for:
  - Companies that do not have their **own software development**.
  - Organizations that use their own software, where **the developers are not staff members.**
  - Companies that have been working for a long time and are **not open to major restructuring.**

Application

Limitations



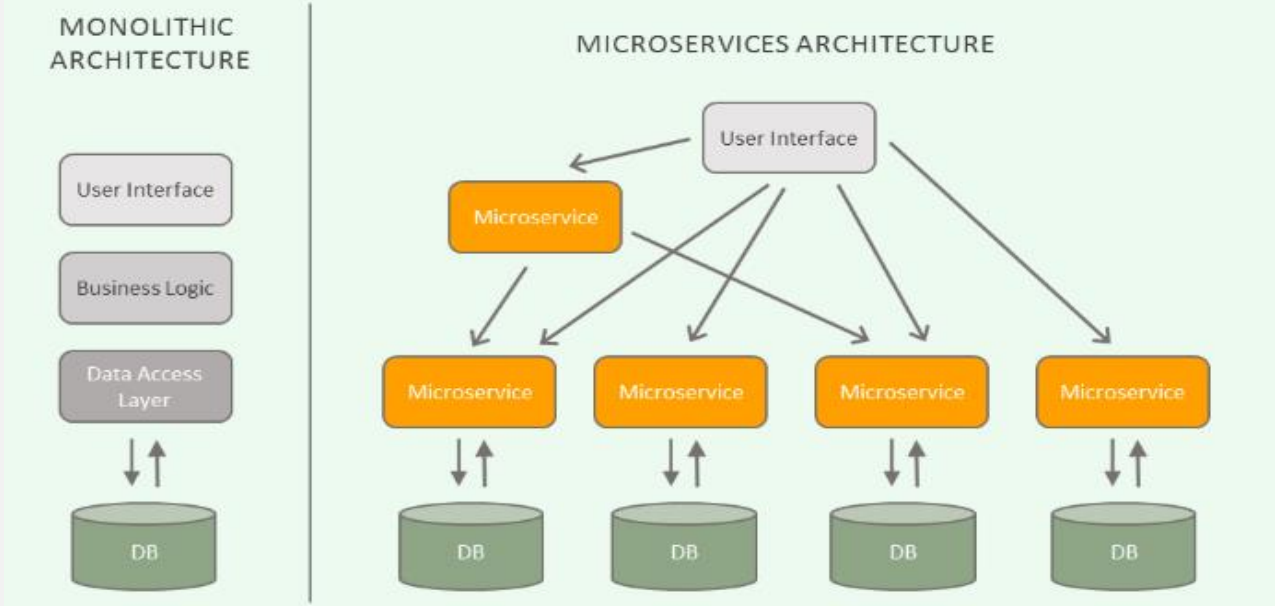
- IT infrastructure and architecture limitations
- Organizations with a monolithic IT architecture.
- Introducing small teams requires the ability to assign a separate area of responsibility to each.



Application

Limitations





COTS


Welcome to Our Site
WELCOME DEAR VISITORS
COTS
Commercial off the Shelf

Application

COTS

➢ The risks of a COTS

- Evolution of the competition:

- It is necessary to have a maximum of flexibility and control, generally inaccessible with commercial software.

- The first piece of advice any serious expert will give you is to get rid of active COTS in the most important areas of your business.

=> Switch to in-house software development.



Application

COTS

- When there's no other option
- Study the installation process in detail, understand what the installer does.
- Then create your own script, replicating the work of the original installer.



Evolving Architecture and Organizational Models

Evolving Architecture and Organizational Models

Evolving Organization

- the difficulties of a traditional IT service:

- Development (Dev) and operations (Ops) are separated in a natural way:
- OPS and support do not go into the intricate details of the system architecture and therefore must ask even the simplest questions to developers.



Evolving Architecture and Organizational Models

Evolving Organization

➢ The difficulties of an IT department:

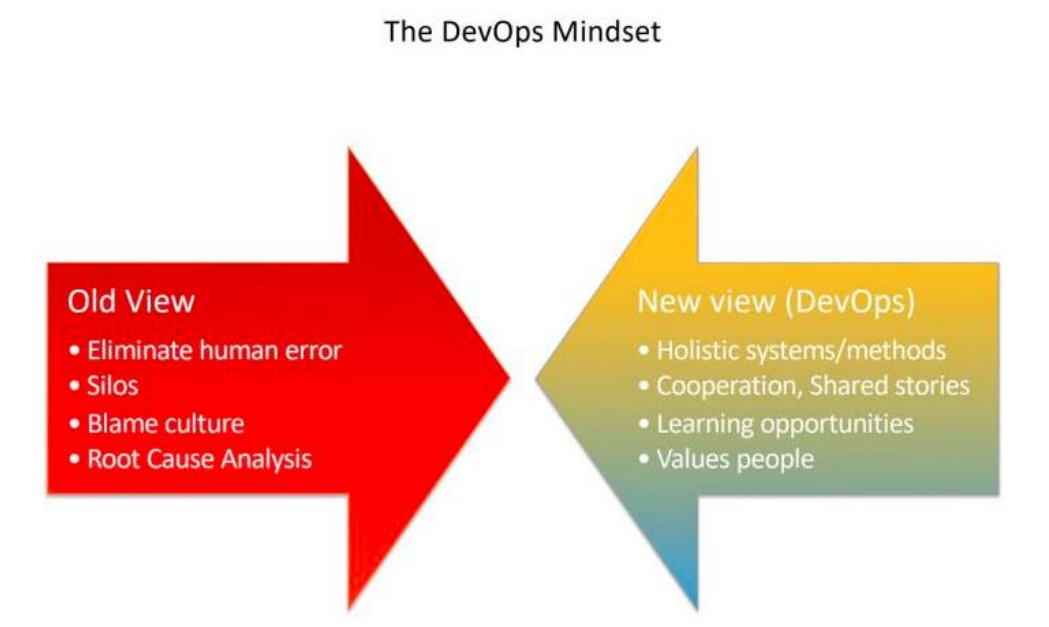
- Any documentation on the IT system quickly becomes obsolete.
- Very few employees understand the IT system in a holistic way, with all the dependencies and constraints; these employees quickly become very valuable, irreplaceable and overburdened.
- Many developers work on the same features simultaneously, each on their own, which requires resources for coordination.



Evolving Architecture and Organizational Models

Evolving Organization

- The need of a mindset change



Evolving Architecture and Organizational Models

Evolving Organization

- The need of a mindset change

Fixed mindsets

- Talents and abilities are innate
- Smart or not smart
- Avoid failures
- Very competitive
- Comfort zone



Growth mindsets

- Talents and abilities are learned and improved
- No fear to fail
- Learn from mistakes
- Like challenges
- Skills can be improved

A growth mindset, allow people and organizations to learn and adapt more quickly.

Evolving Architecture and Organizational Models

Evolving Organization

- The need of a mindset change

Feature	Traditional corporate culture	Startup culture
Management style	Command, authoritarian	Autonomous
Attitude to change	Conservatism	Experiments
Organizational structure	Functional hierarchy	Network
Focus on outputs	Project-oriented	Product-oriented
Change model	Waterfall	Agile, iterative
System architecture	Monolithic, carefully designed	Loosely coupled, microservice
Technology preferences	Patented, proprietary	Open source

Iterative Progression

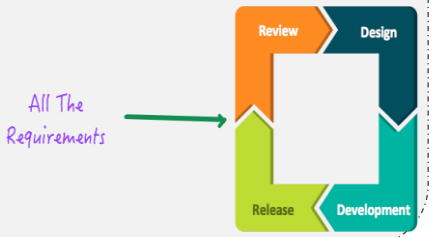
Application

iterative Progression

- Identify systems that are loosely connected to others.
- Use these systems as a pilot to apply the basics of DevOps:

- Value Stream
- Deployment pipeline
- Version control system
- Automated configuration management

- Apply this experience to other systems.
- Starting with simpler cases, you can move on with more importance.



Application

Expirimenting

"Always take on new challenges—even if you are not sure you are completely ready."

Sheryl Sandberg,
COO of Facebook

Application

Embrace failure

- Prepare for failure

In the context of DevOps, failure is not an offense. The teams assume that something is going to go wrong eventually, so they go out of their way to find bugs early and fix them.

- Why ?

Because continual improvement goes hand in hand with failures.

DevOps Culture

Where to start

- OPS culture

- Automate environments
- Open monitoring
- Authorize & supervise access in production
- Share responsibility
- Attend Dev stand ups meetings
- Develop infrastructure tests
- Provide Provisioning API
- Versioning system configurations

DevOps Culture

Where to start

- DEV culture.

- Provide monitoring
- Accompany / carry out the installations
- Share responsibility
- Provide native packaging
- Develop a dashboard of developments
- Bring your voice to the CAB (Change Advisory Board)
- Set up log aggregation
- Automate, Automate, Automate

8 Exam

Exam details

ABOUT EXIN

Published and designed by EXIN. EXIN is the global independent certification institute for professionals in the IT domain. With more than 30 years of experience in certifying the competences of over 2 million IT professionals, EXIN is the leading and trusted authority in the IT market. With over 1000 accredited partners EXIN facilitates exams and e-competence assessments in more than 165 countries and 20 languages. EXIN is co-initiator of the e-Competence Framework, which was set up to provide unambiguous ICT certification measurement principles within Europe and beyond.

EXIN. THE MOST RENOWNED EXAMINATION INSTITUTE FOR IT PROFESSIONALS



- Complete portfolio of Information Management
- Exams in 165 countries
- Exams in 20 languages
- 2 million EXIN-certified professionals
- International network of accredited partners